

Reviewer Pack — p-adic Valuation of Solution Count Modulo p Inversely Propor...

Entry 633842e040fa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 22:03:07 UTC

p-adic Valuation of Solution Count Modulo p Inversely Proportional to DPLL Tree Size

Entry ID: 633842e040fa **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 22:03:07 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): DPLL Search Tree Size

Statement:

For a 3-CNF formula Φ with n variables, the p-adic valuation of the number of satisfying assignments modulo p is $\Theta(\log n)$ if and only if the DPLL search tree size is $\Theta(2^{\{n/2\}})$.

Rationale (proposer's reasoning):

p-adic valuations capture arithmetic structure, which may correlate with the exponential behavior of search trees in DPLL. The valuation reflects solution density modulo p , potentially exposing algebraic regularities in clause interactions.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 058f63ceadaf4915

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def p_adic_valuation(n, p):
        if n == 0:
            return -math.inf
        valuation = 0
        while n % p == 0:
            n //= p
            valuation += 1
        return valuation

    def count_solutions(phi):
        n = len(phi)
        count = 0
        for i in range(2**n):
            if all((i >> j) & 1 or not any(lit % 3 == 0 for lit in clause) for j, clause in enumerate(phi)):
                count += 1
        return count

    def dpll_tree_size(phi, assignment=[]):
        if len(assignment) == n:
            if all(any((assignment[j] >> i) & 1 or not any(lit % 3 == 0 for lit in clause) for j, clause in enumerate(phi))):
                return 1
            else:
                return 0
        count = 0
        for val in [0, 1]:
            assignment.append(val)
            count += dpll_tree_size(phi, assignment)
            assignment.pop()
        return count

    n = random.randint(5, 20)
    phi = [[random.choice([-3*i-1, -3*i+1, 3*i-1, 3*i+1]) for _ in range(random.randint(3, 5))] for i in range(n)]

    solutions_mod_p = count_solutions(phi) % 3
    valuation = p_adic_valuation(solutions_mod_p, 3)
    tree_size = dpll_tree_size(phi)

    metric_name = "p-adic Valuation vs DPLL Tree Size"
    metric_value = (valuation, math.log2(tree_size))
```

```

instances_tested = 1
conjecture_holds = abs(valuation - math.log2(tree_size)) < 0.5
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_valuation = sum(val for val, _ in [r["metric_value"] for r in results]) / len(results)
    mean_tree_size = sum(size for _, size in [r["metric_value"] for r in results]) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean_valuation={mean_valuation} mean_tree_size={mean_tree_size}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93fdd2dc.py", line 78, in <module>

```

    result = run_trial(seed)
    ~~~~~^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93fdd2dc.py", line 52, in run_trial

```

    phi = [[random.choice([-3*i-1, -3*i+1, 3*i-1, 3*i+1]) for _ in range(random.randint(3, 5))] for _ in range(5)]
    ^

```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'i', preventing data collection. Support condition cannot be evaluated without successful trials. | next: Fix the test code to define variable 'i' and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	102931
2	preregistration	ollama_remote	qwen3:8b	0	0	24080
3	novelty	ollama_remote	qwen3:8b	0	0	20742
4	novelty	ollama_remote	qwen3:8b	0	0	13436
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12548
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9766
7	judge	ollama_remote	qwen3:8b	0	0	14570

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 198072 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/633842e040fa.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/633842e040fa.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/633842e040fa.tar.gz (if generated)